

DL for ENG

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **explain** — why neither architecture from L5.1 applies to a sequence
- **write** — the recurrence in one line, and say what the hidden state holds
- **state** — why the gradient dies over time, and how the cell state survives it
- **read** — an attention matrix, and say what it does and does not tell you
- **size** — an LSTM for an engineering series, and choose between it and a GRU
- **judge** — whether a reported improvement is real, from the seed spread

WHAT TODAY ASSUMES

from L5.1	weight sharing, residual connections
from L4.2	the vanishing gradient
not needed	any natural language background

WEIGHT SHARING, A THIRD TIME

A kernel across position, a cell across time, a message function across nodes. One principle, three architectures.

AND THE HONEST SLIDE IS LATE ON

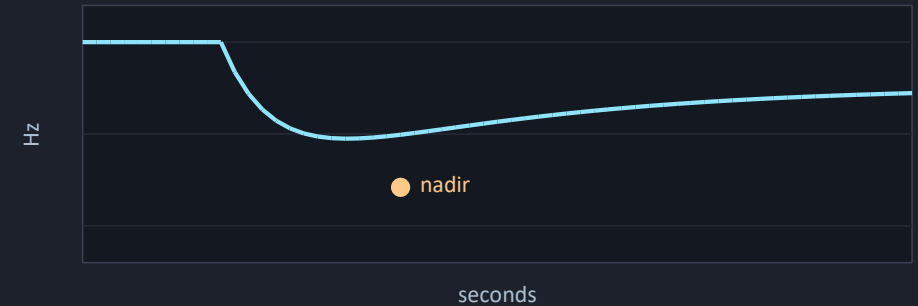
A clear win that turns out not to be there. It may be the most useful slide here.

Sequences Are Most of What Engineering Measures

FOUR EXAMPLES, AND YOU HAVE MET THREE OF THEM

- a **grid frequency trace** — every deviation is a statement about supply and demand
- a **load profile** — a daily cycle, a weekly cycle, and weather in the noise
- a **machine sensor stream** — where a fault announces itself before it arrives
- a **car's speed over a lap** — the JetRacer, once you stop treating frames independently

A GRID FREQUENCY EVENT



WHAT MAKES IT A SEQUENCE PROBLEM

Shuffle the hours of a load profile and you have destroyed the problem, not rearranged it. That is the exact opposite of the six-bus network in L5.1, where renumbering the buses had to change nothing. Order matters here, and any architecture you choose has to know it.

Ex_05 notebook 04 forecasts the next hour of a substation load profile from the last twenty-four. Everything on the honest closing slides is measured on that.

Why Last Hour's Architectures Do Not Apply

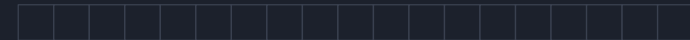
TWO PROPERTIES A SEQUENCE DOES NOT HAVE

- **convolution needs a fixed neighbourhood** — a sequence has no such thing
- **what matters might be one step back** — or a hundred
- **a dense network needs a fixed length** — twenty-four inputs, twenty-four weights
- **real sequences differ in length** — laps, runs, outages
- **so both of last hour's architectures** — decline the problem

FIXED NEIGHBOURHOOD, FIXED LENGTH



an image — every neighbourhood is the same size and shape



three sequences — no shared length, and no shared notion of "the neighbours"

So the requirement is a rule that reads one step at a time and keeps a summary of everything before it. Which is a recurrence — and that is the next slide.

The Recurrent Network, in One Line

A STATE, AND A RULE FOR UPDATING IT

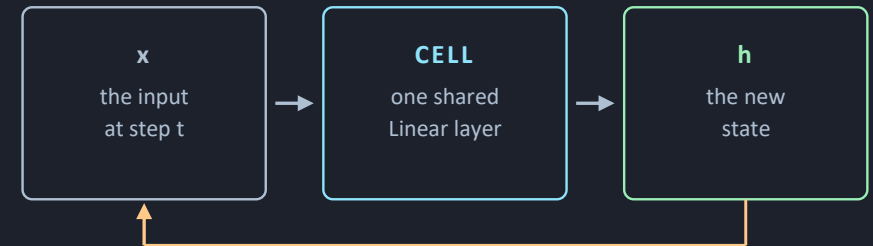
- **keep a vector** — the hidden state
- **at each step** — combine the current input with the current state, and update
- one weight matrix on the state, one on the input, one bias, one activation
- **the state is a lossy summary** — of everything the network has seen so far

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

$$\hat{y} = \mathbf{W}_y \mathbf{h}_T + b_y$$

The recurrence, and the read-out. Everything else today is a repair to the first line.

THE LOOP, DRAWN



the state, fed back into the next step

The feedback arrow is the whole idea, and it is also the whole problem. Information from step one reaches step twenty-four only by passing through every step in between, and each passage squeezes it through a tanh and a matrix.

IN YOUR EXERCISE

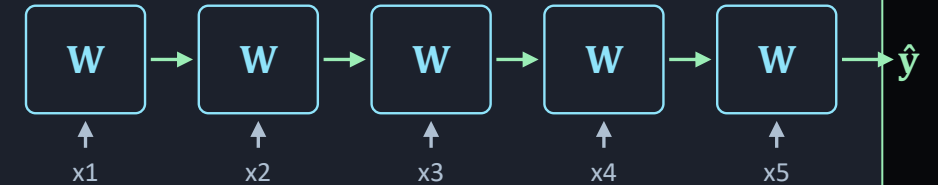
Notebook 04 writes this out in five lines, with no nn.RNN anywhere. The library call hides precisely the line that matters.

Unrolling, and the Third Instance of Weight Sharing

THE SAME PRINCIPLE AS LAST HOUR, ON A THIRD KIND OF STRUCTURE

- **to train it, unroll it** — one copy of the cell per time step
- **unrolling adds no parameters** — every copy is the same matrix
- **it has to be** — you do not know the length in advance
- **weight sharing, for the third time** — kernel across position, now cell across time
- **the principle is unchanged** — same rule everywhere, same weights everywhere

ONE CELL, FIVE COPIES, ONE W



Five boxes, one matrix. The parameter count of a recurrent network is independent of the sequence length — which is exactly why it can take one.

THE SAME IDEA, THREE TIMES

a grid one kernel, every position

a graph one message rule, every node

a sequence one cell, every time step

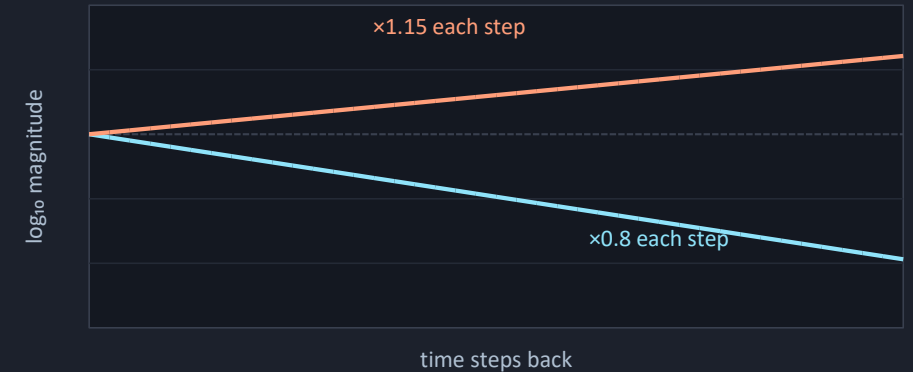
Vanishing Gradients — L4.2 Slide 8, Over Time

THE SAME MECHANISM YOU ALREADY MET, ON A DIFFERENT AXIS

- **L4.2 slide 9, over time** — the gradient is a long product
- **twenty-four steps is a twenty-four layer network** — a thousand steps, a thousand layers
- **and worse** — in a deep network the Jacobians differ; here it is one matrix, repeated
- **below one** — nothing survives beyond about ten steps
- **above one** — it explodes. There is no comfortable middle.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \prod_{t=2}^T \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

GRADIENT MAGNITUDE, GOING BACK IN TIME



ONE OF THE TWO HAS AN EASY FIX

Exploding gradients are cured by clipping: if the gradient norm exceeds a threshold, rescale it. Three lines, and it works. Vanishing gradients have no such trick — the information is gone, not merely large.

The One Half-Lecture That Leaves the Book

SAY THIS TO YOUR STUDENTS AND PUT IT IN THEIR NOTES

Understanding Deep Learning has no recurrent chapter and no LSTM section. Neither does Bishop's 2023 book. The next six slides come from Goodfellow, Bengio & Courville, chapter 10.7 and 10.10 — the 2016 book, and the only source on this course's list that teaches recurrence at all.

WHY BOTH MODERN BOOKS DROPPED IT

Two textbooks, written independently, both decided that recurrence no longer earns a chapter. That is not an accident and it is not an oversight — it is a judgement that attention replaced it.

The judgement is largely right for the problems those books are about: language, vision, anything with a very large dataset.

It is not right for engineering time series in 2026. Machine health, load forecasting, control loops — the deployed code is still full of LSTMs, and so are the papers your supervisors cite.

So you are taught it as a live method rather than as history. You should be able to build one, choose one, and read one that somebody else built.

WHERE EACH PIECE OF TODAY COMES FROM

slides 2 – 3

UDL ch. 12.1 — sequential data

slides 4 – 6

GBC ch. 10.1 – 10.7 — recurrence

slides 8 – 13

GBC ch. 10.10 — the LSTM

slides 14 – 18

UDL ch. 12.2 – 12.4 — attention

slides 19 – 21

UDL ch. 12.6, 12.7, 12.10

Amber rows are not in your main textbook. This is the only week whose reading is split across two books.

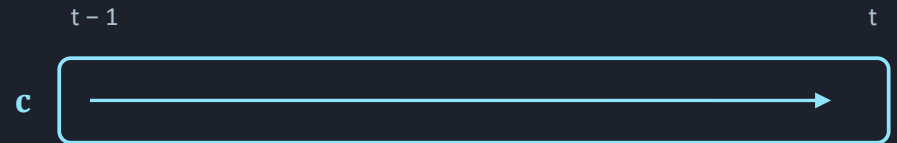
The Cell State, and Nothing Else Yet



A SECOND STATE, UPDATED BY ADDITION

- **add a second state** — the cell state, alongside the hidden one
- **the default operation on it is addition** — not matrix multiplication
- that single change is the whole mechanism
- **on the last slide, information was multiplied at every step** — here it is carried
- **everything else in the LSTM** — controls what goes on and what comes off

THE CONVEYOR BELT



Nothing is drawn on the belt yet, and that is the point of this slide. Left alone, the cell state at step t is the cell state at step $t - 1$. The gradient travelling backwards along it is multiplied by one.

COMPARE WITH L5.1 SLIDE 9

A residual connection is a path through a deep network on which the gradient is multiplied by one. The cell state is a path through time on which the gradient is multiplied by one. Same fix, same reason, eighteen years apart — and the LSTM came first, in 1997.

Two Gates: What to Drop, What to Admit

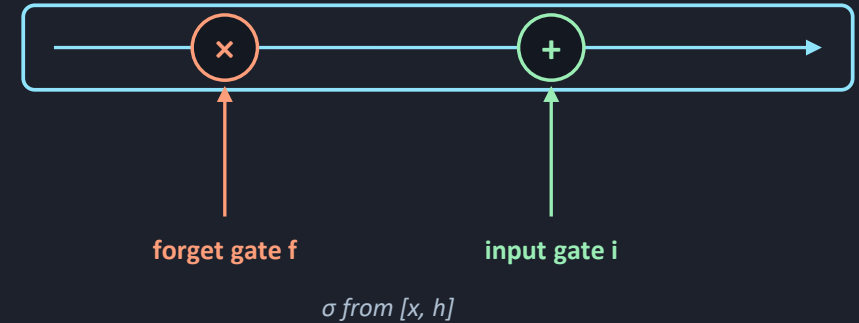


A GATE IS A SIGMOID AND A MULTIPLICATION

- a **gate** — a vector between zero and one, from a sigmoid layer
- the **forget gate** — multiplies the belt. How much of each element to keep.
- the **input gate** — decides how much of a proposal to add
- the **candidate** — is proposed by a tanh layer
- both operations are **elementwise** — the belt is never mixed by a matrix

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_f)$$

THE BELT, NOW WITH TWO CONTROLS



Multiply, then add. The forget gate decides what leaves the belt and the input gate decides what joins it, and both decisions are made afresh at every step from the current input.

THE BIAS THAT MATTERS

Initialise the forget gate's bias to one. A gate that starts near one remembers by default, and the network learns when to forget rather than learning to remember.

The Output Gate, and the Complete Unit

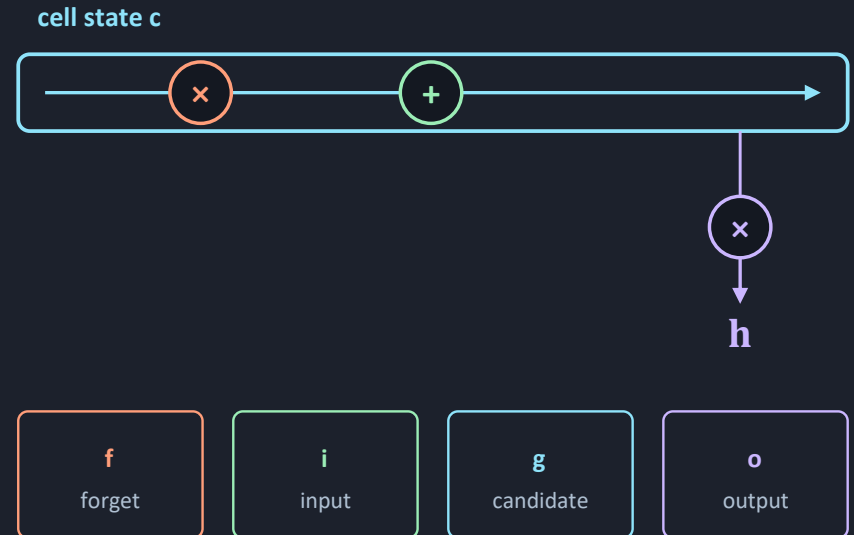


WHAT THE REST OF THE NETWORK IS ALLOWED TO SEE

- **the output gate** — how much of the cell state is exposed as this step's output
- **the separation matters** — the cell state is memory, the hidden state is what it says
- **so it can hold something for fifty steps** — and mention it only at the end
- **a plain recurrent network cannot** — its state is both memory and output
- **four small layers, three of them gates** — and every LSTM you meet is this

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

THE FULL UNIT



all four computed from the same $[x, h]$ pair, by one Linear layer of width $4H$

Four gates from one matrix product is how every real implementation does it, and it is what you will write in notebook 04 — one Linear producing four times the hidden width, then sliced.

The Six Lines, and What Each Buys You

THE WHOLE ARCHITECTURE, WRITTEN OUT

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_f)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_i)$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_g[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_g)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_o)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Three sigmoids, one tanh, one update, one read-out. Notice that the first four lines are the same layer with different weights — which is why they are computed as one matrix product in every implementation.

WHAT EACH GATE BUYS

forget f	a deliberate reset — the shift changed
input i	ignore an irrelevant step entirely
candidate g	what would be written, if admitted
output o	hold a memory without exposing it

THE COST OF ALL THIS

Four gates means four weight matrices, so an LSTM has roughly four times the parameters of a plain recurrent cell with the same hidden width. In notebook 04 that is 1,169 against 305 — and about six times the wall-clock training time.

Why the Gradient Survives

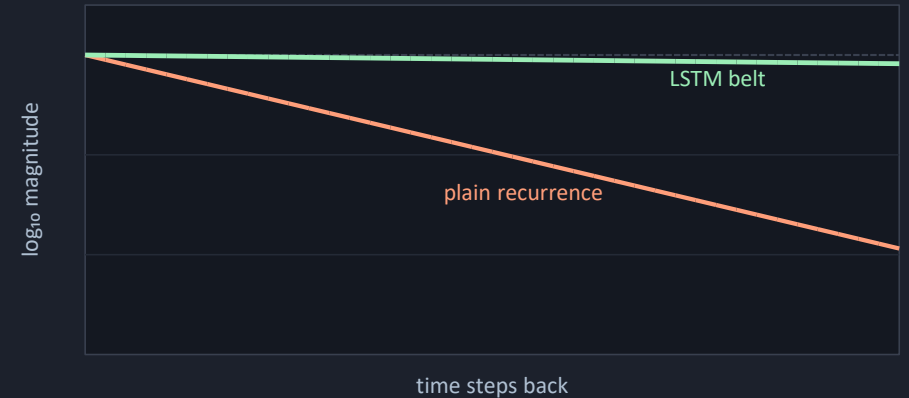
THE ONE DERIVATIVE THAT EXPLAINS THE ARCHITECTURE

- **differentiate the cell update** — the leading term is the forget gate, elementwise
- **compare with slide 6** — a full Jacobian, raised to a power
- **the network controls its own gradient path** — forget gate near one to remember
- not a proof it will learn long dependencies
- a proof that nothing in the architecture prevents it

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t \quad \mathbf{f}_t \approx 1 \Rightarrow \text{gradient survives}$$

The whole of Hochreiter and Schmidhuber's 1997 argument, in one line.

THE SAME PLOT, WITH A GATE AT 0.99



THE HONEST QUALIFIER

The gradient still passes through the gates themselves, and a long product of forget gates below one still decays. The LSTM extends the reach by orders of magnitude; it does not make it infinite.

Using One, and the Variants You Will Meet

THE PRACTICAL SLIDE — WHAT TO DO ON MONDAY

- **hidden width first** — sixteen to a hundred and twenty-eight covers most engineering series
- **then depth** — two stacked layers is usually the most that helps
- **bidirectional only if you have the whole sequence** — offline yes, control no
- **and know the GRU** — two gates instead of three, often as good and quicker

SENSIBLE DEFAULTS

hidden width	32 — grow only if underfitting
layers	1, or 2 at the most
clip the gradient	always, at norm 1
forget bias	initialise to 1

MEASURED IN EX_05 NOTEBOOK 04

plain recurrent	305 params · 0.00127 MSE
LSTM	1,169 params · 0.00079 MSE
perceptron	417 params · 0.00079 MSE

The LSTM repaired the recurrent network's error — and landed exactly where a dense layer already was.

Stop Carrying the Information

THE SENTENCE THE SECOND HALF OF THIS LECTURE IS BUILT ON

Every problem on the last ten slides came from making information travel step by step. So do not make it travel. Give every position a direct path to every other position, and compute all of those paths at once.

WHAT RECURRENCE COSTS

Path length. Information from step one reaches step T through $T - 1$ transformations, and each one can lose it.

Sequential computation. Step t cannot be computed until step $t - 1$ is finished, so a modern parallel processor sits idle. In notebook 04 the recurrent models take five to thirty seconds where the dense model takes a third of one.

A fixed-size bottleneck. Everything the model knows about the past is squeezed into one hidden vector, whatever the sequence length.

None of these is a bug in the implementation. All three are properties of the recurrence itself.

WHAT ATTENTION COSTS INSTEAD

Path length one, between any two positions, regardless of how far apart they are.

One parallel operation. Every position is computed simultaneously, which is why transformers train on hardware that recurrent networks could never saturate.

No bottleneck: the model keeps a representation per position rather than one summary for all of them.

The bill arrives as memory and compute. Comparing every position with every other is quadratic in the sequence length — double the sequence and you quadruple the cost. That is the trade, and it is the reason long-context models are hard.

The recurrent network answers "what do I remember?". Attention answers "what should I look at?" — and the second question is easier.

Queries, Keys and Values

ONE EQUATION, AND IT IS A WEIGHTED AVERAGE

- **three vectors per position** — query, key, value, from three learned matrices
- **compare each query with every key** — a dot product. Large where they match.
- **softmax the scores** — and take that weighted average of the values
- **divide by the square root of the width** — or the softmax saturates and gradients vanish
- **the learned part is only the three projections** — the comparison has no parameters

$$\mathbf{Q} = \mathbf{XW}_q \quad \mathbf{K} = \mathbf{XW}_k \quad \mathbf{V} = \mathbf{XW}_v$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d}}\right)\mathbf{V}$$

THE FIVE STEPS

embed one vector per position

project into Q, K and V

score every query against every key

scale, softmax divide by $\sqrt{d}$, then normalise

average weighted sum of the values

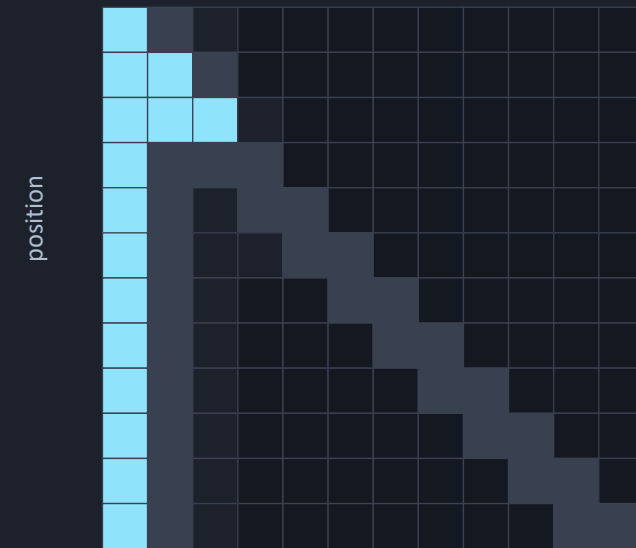
Five steps, no recurrence, and every position computed at the same time. In notebook 04 this is nine lines of PyTorch with no library attention call in sight — write it once and it stops being mysterious.

The Attention Matrix Is a Picture

WHAT THE SOFTMAX ACTUALLY PRODUCES

- **the scores form a square matrix** — one row per position, each summing to one
- **and it is readable** — the one thing attention gives that a dense layer does not
- **in your own notebook** — the model attends to the same hour yesterday
- **nobody told it to** — the daily cycle is the strongest signal and it found it
- **two warnings** — weights say which values were used, not why, and they are not a causal claim

MEAN ATTENTION, HELD-OUT WINDOWS



Bright is heavy. The bright left-hand columns are the start of the window — the same hour yesterday. The bright diagonal is the last few hours. Those are the two things a forecaster should use, and the model found both.

Multiple Heads, and Telling It About Time

TWO ADDITIONS, AND THE SECOND IS NOT OPTIONAL

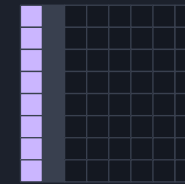
- **one attention operation** — expresses one notion of relevance
- **several heads** — several notions, in parallel
- **and the cost is nothing** — each head gets a slice of the width, not a copy
- **now the part that is not optional** — permute the positions and the output permutes
- **in L5.1 that was the argument for graph networks; here it is a defect** — so add position

$$\text{MultiHead}(\mathbf{X}) = [\mathbf{Z}_1, \dots, \mathbf{Z}_H] \mathbf{W}_O$$

THREE HEADS, THREE VIEWS



recent hours



yesterday



a fixed lag

Different heads, different patterns of relevance, all computed at once.

THE PROPERTY THAT FLIPPED SIGN

L5.1 slide 16: permutation equivariance is why a graph network is right for a power network. Here the same property is wrong, and the positional encoding is the patch. The same mathematics can be a feature or a defect — it depends entirely on what your data means.

The Transformer Layer

FOUR PARTS, AND YOU HAVE ALREADY MET TWO OF THEM

- a **transformer layer** — multi-head attention, then a small network per position
- with residual connections and normalisation around both
- **two of the four parts came from L5.1** — you should recognise them on sight
- **stack it** — six for the original translation model, ninety-six for a large one
- **nothing else changes with scale** — only the count

$$\mathbf{X} \leftarrow \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X})) \quad \mathbf{X} \leftarrow \mathbf{X} + \text{MLP}(\text{LN}(\mathbf{X}))$$

ONE LAYER, TOP TO BOTTOM



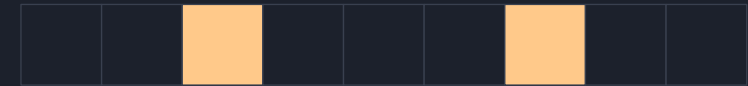
The attention is the only place where positions talk to each other. Everything else in the layer treats each position on its own — which is precisely why the whole thing parallelises.

Encoders — BERT

READ THE WHOLE SEQUENCE, THEN ANSWER ABOUT IT

- **encoder-only** — every position sees every other, in both directions
- **pre-trained by hiding about fifteen per cent** — and asking it to reconstruct
- **then a small head, fine-tuned on your task** — with far fewer labels
- **for an engineer the shape is the point** — masked reconstruction on unlabelled sensor data
- **which is the self-supervision idea from L3.2** — with an architecture attached

MASKED RECONSTRUCTION



two positions hidden, both predicted from the rest



encoder

bidirectional attention

head

your task, fine-tuned

Both directions are available because there is nothing to predict in sequence — the model is not generating, it is reading. That single decision is the whole difference from the next slide.

WHAT SLIDES 19 TO 21 ARE FOR

Enough to read a paper and follow a conversation about one. Not enough to build one, and not trying to be.

Decoders — GPT-Style Models

PREDICT THE NEXT ONE, FOREVER

- **decoder-only** — position i attends only to positions up to i
- **trained by next-token prediction** — every position is a training example
- **generation is the same operation, forwards** — predict, append, predict
- **the base model is not the assistant** — instruction tuning and RL come after
- **which is L6.2** — and it is where the two halves of this course meet

THE CAUSAL MASK



row i attends only to columns $\leq i$

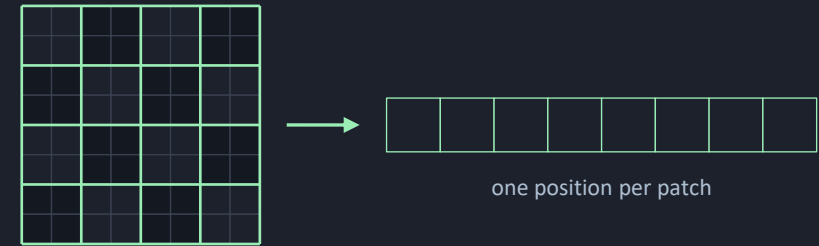
Nothing above the diagonal. A position cannot see its own future, so the same forward pass gives a valid prediction at every position at once — one pass, hundreds of training signals.

Vision Transformers

AN IMAGE AS A SEQUENCE OF PATCHES

- **cut the image into patches** — sixteen by sixteen, flattened and projected
- **then it is a sequence** — and the transformer applies unchanged
- **on very large datasets** — it outperforms a convolutional network
- **the reason it needs them** — is L5.1's argument, running backwards
- **so on a few hundred inspection images, use the CNN** — that is engineering, not caution

PATCHES, THEN A SEQUENCE



The picture is now a sequence, the positional encoding is the only thing that remembers where each patch came from, and every patch can look at every other in the first layer — which a convolutional network cannot do until it is deep.

THE ENGINEERING READING

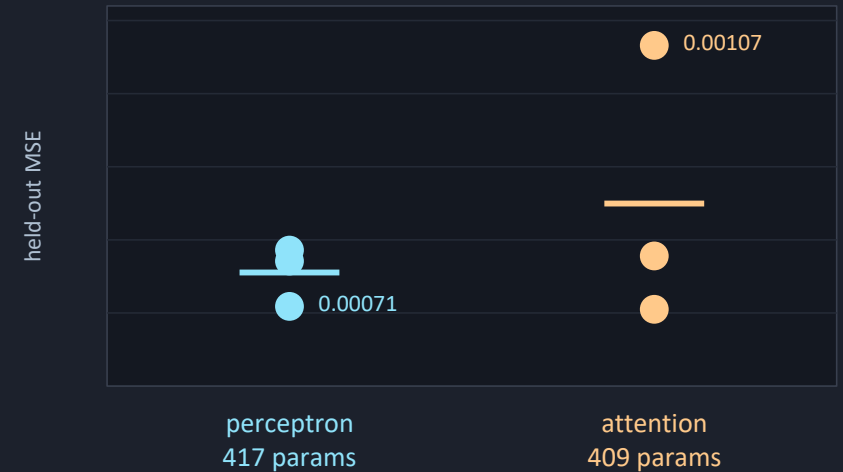
Vision transformers are the right choice when you have millions of images. In this course you have hundreds, and L5.1's convolution wins on every one of them.

A Ten Per Cent Win That Is Not There

MEASURED IN YOUR OWN EXERCISE, ON THREE SEEDS

- **in notebook 04** — attention comes out nearly twenty per cent ahead on held-out error
- **then retrain from three seeds** — and plot the spread
- **the bands overlap** — comfortably
- **the margin was a property of seed zero** — not of the architecture
- **state it carefully** — attention is not worse; the evidence did not support the claim

THREE SEEDS EACH — THE SPREAD IS THE STORY



The distributions overlap completely. A difference smaller than the seed spread is not a result, and one run of each is not a comparison. Three seeds cost you four minutes here.

An Excellent Error Hiding a Failure

THE CHECK THAT IS ALWAYS SKIPPED

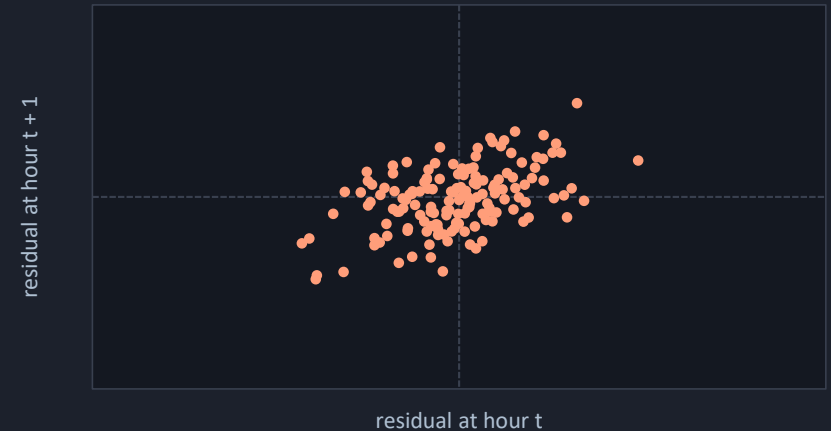
The best model in that notebook has a held-out error four times better than persistence. The loss curves are smooth and flat. All four architectures agree on the number, which makes it look like a floor set by the noise.

Then plot each residual against the next one. The scatter leans upwards, and the lag-one autocorrelation of the residuals comes out at nought point five.

That is a failure, and it is the most useful thing in the notebook. Half of the error at the next hour is a deterministic function of the error at this hour — so it was predictable, and the model did not predict it. By construction, in fact: the generator's noise is an AR(1) process with a coefficient of nought point seven five.

Nothing else told you this. Not the mean squared error, not the loss curves, not the comparison between architectures. Only the residual check, and it is two lines of code. We left that failure in the notebook rather than tuning it away, because a diagnostic that always passes teaches nobody anything.

RESIDUAL AT t AGAINST RESIDUAL AT $t + 1$



lag-1 autocorrelation: 0.50

If the model had extracted everything a twenty-four hour window contains, this would be a formless cloud. It is not a cloud. It leans, and the lean is free accuracy that nobody collected.

Failure Modes

SHUFFLING BEFORE SPLITTING

Two overlapping windows share twenty-three of their twenty-four values. Shuffle first and your test set is nearly identical to your training set — the model has seen the future of its own evaluation, and your held-out number is fiction. Split in time order.

NO POSITIONAL ENCODING

Attention is permutation equivariant. Forget the positional encoding on a time series and the model is literally unable to tell first from last. It will still train, and it will still report a loss.

BIDIRECTIONAL ON A LIVE SIGNAL

A backward pass reads the future. Excellent for offline fault analysis, invalid for anything that runs in real time — and the offline metrics look wonderful right up until deployment.

WHAT TO CHECK, IN ORDER OF COST

free	is the split in time order?
free	compare against persistence
cheap	plot the residual autocorrelation
cheap	rerun from three seeds
real work	vary the window length
real work	test on a different site or machine

Lines one to four cost you under ten minutes together, and between them they catch the leaked split, the meaningless baseline, the structured residual and the imaginary architecture win. Everything on the last two slides came from lines two to four.

Ex_05 Notebook 04 — Four Models, One Problem

EVERYTHING ON TODAY'S SLIDES, WRITTEN OUT BY HAND

Notebook 04 forecasts the next hour of substation demand from the last twenty-four. Forty days of data, generated on your own machine, with a daily cycle, a weekly cycle and correlated noise.

You will write four models: the recurrence from slide four, the LSTM cell from slides eight to eleven, single-head self-attention from slide fifteen, and a plain perceptron on the flattened window. There is no `nn.LSTM` in the notebook and no library attention call — five lines each, and those five lines are the entire content of the architecture.

Then the two comparisons on slides twenty-two and twenty-three: three seeds each, and the residual autocorrelation. Expect the perceptron to match everything else, and be ready to say why that is a statement about the data rather than about attention.

Two baselines are computed before any model is trained: persistence, which predicts the last value, and the training mean. Persistence is seven times better than the mean here. Any model that does not beat persistence has learned nothing an operator did not already know.

WHAT YOU WILL BUILD

baselines	persistence and the mean
SimpleRNN	the recurrence, in a for loop
SimpleLSTM	four gates from one Linear
SelfAttention	Q, K, V and one softmax
a perceptron	24 in, 16 hidden, 1 out

THE QUESTION THAT CARRIES THE MARKS

You will have four models within a factor of two of each other. Which would you deploy on a substation controller, and which single number from your table decided it? "The most accurate one" is not an answer here.

Key Takeaways

1

Weight sharing, a third time

One kernel per position, one rule per node, one cell per time step. The same principle on a third kind of structure — and it is what frees the parameter count from the sequence length.

2

Recurrence fails by construction

The gradient through time is a power of one matrix, so it either vanishes or explodes. L4.2 slide 8, over time rather than over depth.

3

The LSTM buys an additive path

A cell state updated by addition, and three gates controlling it. The derivative along the belt is the forget gate, not a Jacobian. Sourced from Goodfellow chapter 10.10, not from UDL.

4

Attention removes the carrying

Every position reads every other in one parallel step. The bill is quadratic cost and a positional encoding, because the mechanism does not know about time until you tell it.

5

Fashion is not selection

Your own three-seed comparison dissolves attention's seed-zero win, and your residual check finds structure an excellent MSE hid. Measure, then choose.

Next — L6.1: loss functions and gradients. How any of these architectures is actually fitted.

Questions

TEN TO TEST WHETHER TODAY LANDED

- **why does a convolution not apply to a sequence** — and why does a dense layer not either?
- **what does the hidden state hold** — in one sentence?
- why does the gradient die over time, and why is it worse than in depth?
- **what does the cell state change** — and why is addition the key?
- **query, key and value** — which of the three is learned?
- what does an attention weight tell you, and what does it not?

AND FOUR MORE

- | | |
|--------------|--|
| seven | why divide by the square root of the width? |
| eight | why is positional encoding not optional? |
| nine | why does a vision transformer need more data than a CNN? |
| ten | a model wins by ten per cent. What do you ask first? |

THE ONE WITH NO SETTLED ANSWER

One sequence, three thousand samples, a deadline. LSTM or transformer, and what would change your mind?

BEFORE L6.1

Run notebook 04. Bring the three-seed plot.

References and Further Reading

THE TWO BOOKS, AND WHICH HALF EACH ONE COVERS

Prince, Understanding Deep Learning — chapter 12 in full: 12.2 for dot-product self-attention, 12.3 for multi-head and positional encoding, 12.4 for the transformer layer, then 12.6, 12.7 and 12.10 for BERT, GPT and vision transformers. Chapter 11.1 for sequential data. There is no recurrent chapter, and searching for one wastes an hour.

Goodfellow, Bengio & Courville, chapter 10 — sections 10.1 and 10.2 for the recurrence and unrolling, 10.7 for the vanishing gradient over time, 10.10 for the LSTM and the GRU. This is the only source on the course list that teaches recurrence, and everything on slides 4 to 13 comes from it.

The same search trap as last hour applies: that chapter is called "Recurrent and Recursive Nets", and recursive there means tree-structured composition, not anything you met in L5.1.

THE PRIMARY SOURCES

Hochreiter & Schmidhuber (1997), Long short-term memory — the cell state and the gates, and it is more readable than its reputation.

Vaswani et al. (2017), Attention is all you need — the transformer layer on slide 18, in eleven pages.

Dosovitskiy et al. (2021) — an image is worth 16×16 words, which is slide 21.

READ IN THIS ORDER

before Ex_05 nb04 GBC ch. 10.1 – 10.2 and 10.10

before Ex_05 nb04 UDL ch. 12.1 – 12.3

if slide 18 was fast UDL ch. 12.4 — it is short

for the history Hochreiter & Schmidhuber (1997)

for the turn Vaswani et al. (2017)

before L6.1 UDL ch. 5.1 – 5.3

This is the only week of Part 1 whose reading is split across two books. The split is on the slide numbers: four to thirteen are Goodfellow, fourteen to twenty-one are Prince.

NEXT

L6.1 — Loss Functions and Gradients

You now have every architecture this course uses — dense, convolutional, graph, recurrent and attention. Block six is about the thing that has been quietly assumed underneath all five of them: where the loss comes from, and how the parameters actually move.

BEFORE THE NEXT SESSION

read	UDL ch. 5.1 – 5.3
run	Ex_05 notebooks 04 and 05
bring	your three-seed spread
think about	which model you would deploy, and on what number

Bring the seed spread. Between L5.1's permutation numbers and this week's three seeds you now have two measured cases where the interesting architecture did not win — and knowing that is worth more than either architecture.